

Setup & Installation Guide: GridGuard AI

Notice for Hackathon Judges & Evaluators:

GridGuard AI has been engineered for zero-friction evaluation. You can run the entire multi-tier platform (Frontend, Backend Gateway, and AI Risk Engine) on any standard laptop without external database or cloud credentials. If PostgreSQL or watsonx.ai credentials are not supplied, the platform automatically activates its built-in resilient fallback engine, providing full interactive access to all 12 substations, telemetry charts, AI risk models, and authentication workflows.

1. Prerequisites

Before installing, ensure your development machine meets the following minimum requirements:

TOOL	MINIMUM VERSION	RECOMMENDED VERSION	VERIFICATION COMMAND
Node.js	v18.0.0+	v20.x or v22.x (LTS)	node -v
npm	v9.0.0+	v10.x+	npm -v
Python	3.10+	3.11.x	python --version or python3 --version
Git	2.30+	Latest	git --version
PostgreSQL (Optional)	14+	15+ or Docker	psql --version
IBM Cloud Account (Optional)	-	Active with watsonx.ai	N/A (Optional)

2. Environment Variables

The repository includes pre-configured environment templates. Create your local `.env` files using the instructions below.

A. Root & AI Engine Environment (`src/.env`)

Copy `src/.env.example` to `src/.env`:

```
# In Windows PowerShell / CMD / Git Bash / macOS / Linux:
cp src/.env.example src/.env
```

Contents of `src/.env`:

```
# IBM watsonx.ai Configuration (Optional for cloud LLM inference)
WATSONX_API_KEY=your_api_key_here
WATSONX_PROJECT_ID=your_project_id_here
WATSONX_URL=https://us-south.ml.cloud.ibm.com

# Database Connection (Optional – uses mock engine if offline)
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/gridguard_db

# Application Runtime Ports
APP_PORT=8001
APP_ENV=development

# Notifications & Alerts (Optional)
SLACK_WEBHOOK_URL=https://hooks.slack.com/services/your/webhook/url
```

B. Backend REST API Environment (`src/backend/.env`)

Copy `src/backend/.env.example` to `src/backend/.env`:

```
cp src/backend/.env.example src/backend/.env
```

Contents of `src/backend/.env`:

```
PORT=5000
NODE_ENV=development
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/gridguard_db
CORS_ORIGIN=http://localhost:3000
```

Environment Variable Reference Table

VARIABLE	TARGET SERVICE	PURPOSE	REQUIRED FOR LOCAL DEMO?
<code>PORT</code>	Backend	HTTP port for the Node.js API gateway (default: <code>5000</code>).	No (Defaults to <code>5000</code>)
<code>APP_PORT</code>	AI Engine	HTTP port for FastAPI Python service (default: <code>8001</code>).	No (Defaults to <code>8001</code>)

VARIABLE	TARGET SERVICE	PURPOSE	REQUIRED FOR LOCAL DEMO?
DATABASE_URL	Backend / AI	PostgreSQL connection URI.	No (Resilient fallback activates automatically if DB is unreachable)
WATSONX_API_KEY	AI Engine / MCP	IBM Cloud IAM API Key for watsonx foundation models.	No (Optional)
WATSONX_PROJECT_ID	AI Engine / MCP	Target watsonx project identifier.	No (Optional)
CORS_ORIGIN	Backend	Allowed origin for frontend requests (http://localhost:3000).	No (Pre-configured)

3. Step-by-Step Installation

Clone the repository and install dependencies for all three tiers:

```
# 1. Clone the repository
git clone https://github.com/harshil5506/bob-ai-hackathon-GridGaurdAI.git
cd bob-ai-hackathon-GridGaurdAI

# 2. Install Frontend dependencies (React + Vite + Recharts)
cd src/frontend
npm install

# 3. Install Backend dependencies (Node.js + Express)
cd ../backend
npm install

# 4. Install AI Engine dependencies (Python FastAPI + scikit-learn + Pydantic)
cd ../ai-engine
# Optional: Create and activate a Python virtual environment
# python -m venv venv
# On Windows: .\venv\Scripts\activate
# On Linux/macOS: source venv/bin/activate
pip install -r requirements.txt
```

4. Running the Application

GridGuard AI consists of three lightweight microservices that run concurrently:

SERVICE	PORT / URL	PURPOSE
Frontend Web App	http://localhost:3000	SCADA Control Console
Backend API	http://localhost:5000	REST Gateway & SCADA
AI Risk Engine	http://localhost:8001	IEEE Failure Predictor

Open three terminal windows (or run in background tabs):

Terminal 1: Frontend Development Server

```
cd src/frontend
npm run dev
```

Output will show: VITE v5.x ready in ... ms → Local: http://localhost:3000/

Terminal 2: Backend REST Gateway

```
cd src/backend
npm start
```

Output will show: GridGuard Backend listening on port 5000

Terminal 3: AI Analytics Engine

```
cd src/ai-engine
python main.py
```

Output will show: Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)

5. Verification & Quick Demo Walkthrough

A. Health Check Verification

Verify all services are responsive using your browser or terminal:

```
# Verify AI Engine health (Returns status: ok, weights, version):  
curl http://localhost:8001/health  
  
# Verify Frontend HTTP response:  
curl -I http://localhost:3000/
```

B. Interactive Control Room Demo (For Judges)

1. **Open the Sign In Terminal:**

Navigate to `http://localhost:3000/signin`.

2. Observe the **interactive generating station canvas background**: high-voltage electrical arcs crackle across gantry towers, multi-phase catenary power lines pulse with electricity packets, and cooling tower steam rises continuously.

3. Move your mouse across the screen to trigger **interactive electrical discharge spark bursts**.

4. Note the floating **SCADA Generation Telemetry HUD** (`2,845 MW`, `59.998 Hz`, `502.4 kV`).

5. **1-Click Pre-Loaded Demo Login:**

6. Click the **"CHIEF CONTROLLER"** button under *Pre-Loaded Demo Credentials*.

7. The system automatically populates the certified operator identity:

- **Email:** `dispatcher.alpha@gridguard.utility`
- **Clearance Key:** `Omega-7-HighVoltage`

8. Click **"AUTHENTICATE OPERATOR"**.

9. **Operator Profile Studio** (`/profile`):

10. After authentication, you are immediately routed to the **Operator Profile**.

11. Review the operator avatar with rotating radar pulse, clearance level badge (**LEVEL 3 // CHIEF CONTROLLER**), and operational SCADA stats (**ONLINE, 24 FLEETS, 98.4% AI ACCURACY, NERC-CIP**).

12. Test the **Demo Role Switcher** buttons to instantly toggle clearance between Chief Controller and Field Dispatcher.

13. Review the immutable **Operator Audit Trail** logging field orders and thermal acknowledgments.

14. **Common Grid Dashboard** (`/`):

15. Click **"DISPATCH CONSOLE"** or **"GRID"** in the top header.

16. Explore the interactive grid map featuring **12 high-voltage substations and transformers** color-coded by real-time risk severity.
 17. Review the **Ranked Priority Table** sorting assets by failure probability and downstream customer impact.
 18. **Specific Asset Deep-Dive Diagnostics:**
 19. Click on **"SUB-001 (Northside Substation Alpha)"**.
 20. Examine real-time sensor trends: Critical temperature (112°C), high partial discharge (420 pC), and degraded oil quality.
 21. Review the AI failure prediction window ($P < 14\%$) and the **"Dispatch Repair Crew"** action button.
 22. **Simulated Password Recovery (/forgot-password):**
 23. Navigate to `http://localhost:3000/forgot-password`.
 24. Enter `dispatcher.alpha@gridguard.utility` and click **"DISPATCH RESET CODE"**.
 25. Observe the simulated incoming email card showing the delivered 6-digit code with a **1-click auto-paste** action, followed by digit-by-digit OTP verification and key rotation.
-

6. Running Unit & Integration Tests

To run the automated test suite:

```
# 1. Test Python AI Engine (IEEE thresholds, risk scoring, data fusion):
cd src/ai-engine
python test_ai_engine.py

# 2. Test Backend API endpoints:
cd src/backend
npm test

# 3. Test Frontend TypeScript compilation & production build:
cd src/frontend
npx tsc --noEmit
npm run build
```

7. Troubleshooting Common Issues

SYMPTOM / ERROR	ROOT CAUSE	EXACT SOLUTION
<code>EADDRINUSE: address already in use :::3000</code> (or <code>:::5000</code> / <code>:::8001</code>)	Another process or previous run is occupying the port.	Find and terminate the process occupying the port: Windows: <code>netstat -ano findstr :3000</code> then <code>taskkill /PID <PID> /F</code> Linux/macOS: <code>lsof -ti:3000 xargs kill -9</code>
<code>password authentication failed for user "postgres"</code>	Local PostgreSQL service is either not started or configured with different credentials.	No action needed! The frontend and backend include an automatic fallback mechanism that serves realistic high-fidelity mock data for all 12 substations, charts, and AI models so you can evaluate the app without a running database.
<code>ModuleNotFoundError: No module named 'fastapi'</code> (or <code>uvicorn</code>)	Python dependencies were installed in a different environment or global path.	Ensure your active Python terminal has dependencies installed: <code>pip install -r src/ai-engine/requirements.txt</code>
<code>npm ERR! code ENOENT</code> when running <code>npm run dev</code>	Terminal is in the repository root rather than the frontend directory.	Change directory to <code>src/frontend</code> : <code>cd src/frontend && npm run dev</code>
Vite opens blank screen or syntax errors in older browsers	Browser lacks modern ES2022+ module support.	Open the app in any modern version of Google Chrome, Microsoft Edge, Firefox, or Safari.
<code>Error: Failed to fetch</code> in developer console	Backend or AI Engine terminal is not currently running.	Ensure all three terminals are active (Frontend on <code>:3000</code> , Backend on <code>:5000</code> , AI Engine on <code>:8001</code>).